

cEOS Guide for SONiC Management Testbed

Overview

This guide provides comprehensive instructions for implementing cEOS (Container EOS) containers as an alternative to virtual machines for SONiC testbed side switches. The containerized approach offers significant improvements in resource utilization and deployment efficiency while maintaining full compatibility with existing testbed configurations.

Why Choose cEOS Containers?

cEOS is EOS with all its agents packed into docker container.

Resource Efficiency

- **Reduced Memory Footprint:** Each cEOS container consumes approximately 1GB of memory compared to 4-8GB required by traditional VMs
- **Enhanced Scalability:** Support for larger network topologies on the same hardware infrastructure
- **Optimized Performance:** Faster startup times and reduced overhead

Operational Benefits

- **Simplified Management:** Containers are automatically created during the topology addition process
 - **No VM Overhead:** Eliminates the need for virtual machine management and maintenance
 - **Default Implementation:** cEOS is default VM type in testbed-cli.sh
-

Getting Started with cEOS

Image Download and Installation

The cEOS image package(2.4GB) is stored in azure blob storage at <https://nexthop.blob.core.windows.net/ceos-images/cEOS64-lab-4.32.5M.tar>

You can download using

```
Shell
az storage blob download \
  --account-name nexthop \
  --container-name ceos-images \
  --name cEOS64-lab-4.32.5M.tar \
  --file cEOS64-lab-4.32.5M.tar \
  --auth-mode key \
  --account-key <STORAGE_ACCOUNT_KEY>
```

VM Type Configuration

When the VM type is configured as cEOS, the system automatically creates Docker-based network neighbors instead of traditional KVM virtual machines. This process occurs during the `testbed-cli.sh add-topo` execution, eliminating the need for separate VM startup procedures.

Standard Deployment Command:

```
Shell
# cEOS is the default configuration, making -k ceos optional
./testbed-cli.sh -t testbed.yaml -m veos add-topo example_testbed
password.txt

# Explicit cEOS specification (identical result)
./testbed-cli.sh -t testbed.yaml -m veos -k ceos add-topo
example_testbed password.txt
```

The resulting cEOS neighbor configuration maintains identical network interface sets compared to vEOS implementations, ensuring seamless compatibility.

Understanding Container Architecture

Dual-Container Design

Each cEOS-based neighbor implementation utilizes a two-container architecture:

1. Network Container

- **Naming Convention:** `net_<group-name>_<vm-name>`
- **Example:** `net_vms1-1_VM0100`
- **Function:**
 - Executes a minimal Debian image
 - Hosts all network interface configurations
 - Provides network namespace foundation

By default, this network container will try to download from the same container registry as the PTF container since it is using the same Ansible variable. If we continue to use an internal registry, we will need to add this container to it. If not this is the type of error that will be seen:

None

```
TASK [vm_set : Wait for creation of net base containers]
*****
      ERROR    task path:
/data/private-sonic-mgmt/ansible/roles/vm_set/tasks/add_ceos_list.yml:131
      ERROR    Monday 23 June 2025  19:48:47 +0000 (0:00:01.618)
0:02:20.754 *****
      ERROR    failed: [ts102_tps_tb3] (item={'failed': 0, 'started': 1,
'finished': 0, 'ansible_job_id': '601414087823.630406', 'results_file':
'/root/.ansible_async/601414087823.630406',
      'changed': True, 'vm_name': 'VM0300', 'ansible_loop_var':
'vm_name'}) => {"ansible_job_id": "601414087823.630406", "ansible_loop_var":
"async_create_result_item",
      "async_create_result_item": {"ansible_job_id":
"601414087823.630406", "ansible_loop_var": "vm_name", "changed": true,
"failed": 0, "finished": 0, "results_file":
"/root/.ansible_async/601414087823.630406", "started": 1,
"vm_name": "VM0300"}, "attempts": 1, "changed": false, "finished": 1, "msg":
"Error pulling image

container-registry.sw.internal.nexthop.ai:443/debian:bookworm - 404 Client
Error for

http+docker://localhost/v1.49/images/create?tag=bookworm&fromImage=container-re
gistry.sw.internal.nexthop.ai%3A443%2Fdebian: Not Found (\\"manifest for
```

```
container-registry.sw.internal.nexthop.ai:443/debian:bookworm not found:
manifest unknown: manifest unknown\""), "results_file":
"/root/.ansible_async/601414087823.630406",
      "started": 1, "stderr": "", "stderr_lines": [], "stdout":
"", "stdout_lines": []}
```

The heavy lifting of setup the containers and the networks are done in task:

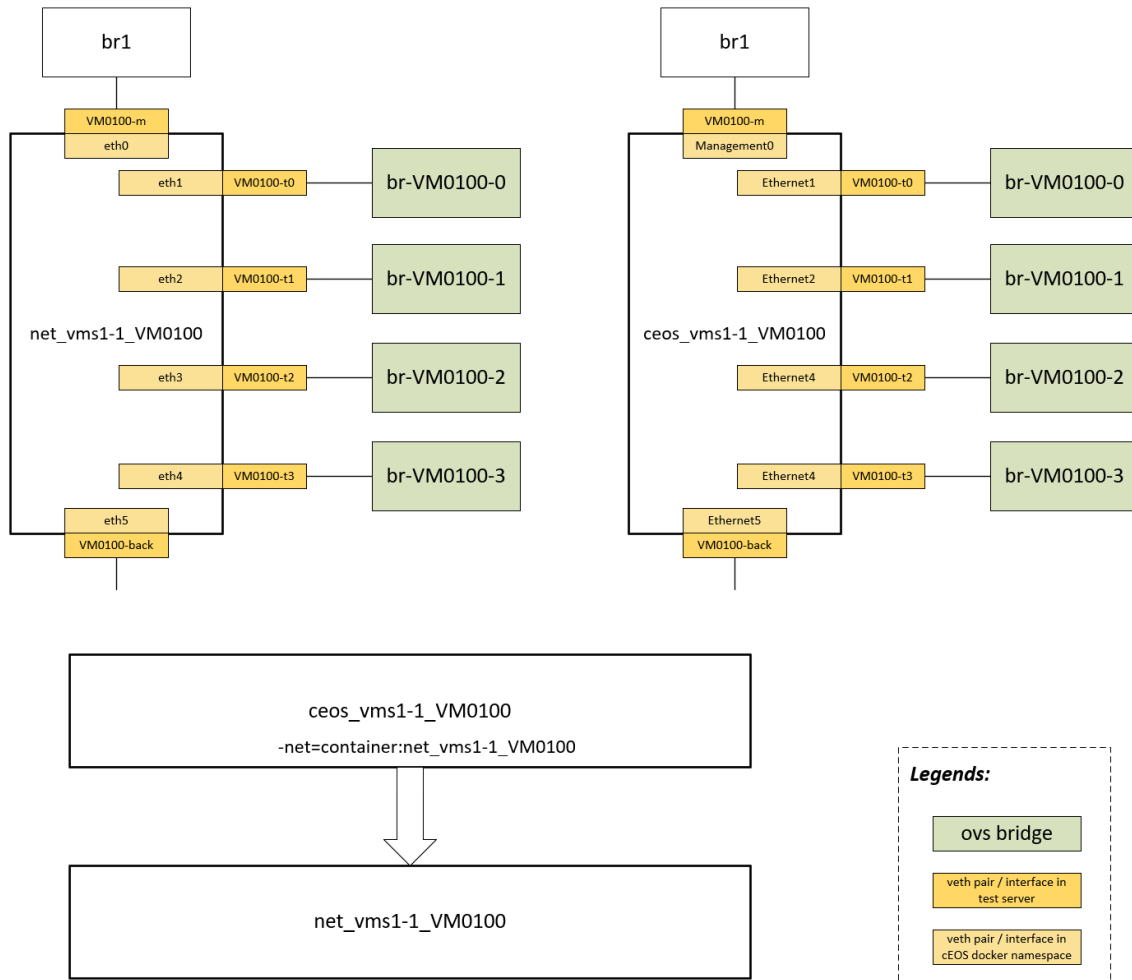
https://github.com/nexthop-ai/private-sonic-mgmt/blob/master/ansible/roles/vm_set/tasks/add_c_eos_list.yml#L143

With the veth pairs created in this python library:

https://github.com/nexthop-ai/private-sonic-mgmt/blob/master/ansible/roles/vm_set/library/ceos_network.py

2. cEOS Container

- **Naming Convention:** `ceos_<group-name>_<vm-name>`
- **Example:** `ceos_vms1-1_VM0100`
- **Function:**
 - Runs the complete cEOS image
 - Operates in shared network namespace mode
 - Inherits all network interfaces from the network container



The cEOS containers also need a eos configuration template for each different topology that its being deployed on. Including Nexthop's standard t1-8 topology. Here is the error condition that pops up if cEOS configuration template is not found for the topology thats trying to be deployed

None

```
<10.20.16.22> SSH: EXEC sshpass -d14 ssh -o ControlMaster=auto -o
ControlPersist=120s -o UserKnownHostsFile=/dev/null -o StrictHostKeyChecking=no
-o ServerAliveInterval=30
```

```

-o ServerAliveCountMax=40 -o StrictHostKeyChecking=no -o
'User="dt"' -o ConnectTimeout=60 -o
'ControlPath="/home/dt/.ansible/cp/bf5ab7d8bd"' 10.20.16.22 '/bin/sh -c ''''(
    umask 77 && mkdir -p "` echo /tmp/.ansible-$USER `"&& mkdir
"` echo /tmp/.ansible-$USER/ansible-tmp-1750775094.6895227-6038-260647376221600
`" && echo
    ansible-tmp-1750775094.6895227-6038-260647376221600="` echo
/tmp/.ansible-$USER/ansible-tmp-1750775094.6895227-6038-260647376221600 `") &&
sleep 0''''''

<10.20.16.22> SSH: EXEC sshpass -d15 ssh -o
ControlMaster=auto -o ControlPersist=120s -o UserKnownHostsFile=/dev/null -o
StrictHostKeyChecking=no -o ServerAliveInterval=30
-o ServerAliveCountMax=40 -o StrictHostKeyChecking=no -o
'User="dt"' -o ConnectTimeout=60 -o
'ControlPath="/home/dt/.ansible/cp/bf5ab7d8bd"' 10.20.16.22 '/bin/sh -c ''''rm
-f -r
/tmp/.ansible-dt/ansible-tmp-1750775094.678367-6031-174129119733354/ >
/dev/null 2>&1 && sleep 0''''''
META: noop
The full traceback is:
Traceback (most recent call last):
  File
"/usr/local/lib/python3.8/dist-packages/ansible/plugins/action/template.py",
line 85, in run
    source = self._find_needle('templates', source)
  File
"/usr/local/lib/python3.8/dist-packages/ansible/plugins/action/__init__.py",
line 1417, in _find_needle
    return
self._loader.path_dwim_relative_stack(path_stack, dirname, needle)
  File
"/usr/local/lib/python3.8/dist-packages/ansible/parsing/dataloader.py", line
341, in path_dwim_relative_stack
    raise AnsibleFileNotFound(file_name=source,
paths=[to_native(p) for p in search])
ansible.errors.AnsibleFileNotFound: Could not find or
access 't1-8-tor.j2'
Searched in:

/data/private-sonic-mgmt/ansible/roles/eos/templates/t1-8-tor.j2

/data/private-sonic-mgmt/ansible/roles/eos/t1-8-tor.j2

/data/private-sonic-mgmt/ansible/roles/eos/tasks/templates/t1-8-tor.j2

```

```

/data/private-sonic-mgmt/ansible/roles/eos/tasks/t1-8-tor.j2

/data/private-sonic-mgmt/ansible/templates/t1-8-tor.j2
/data/private-sonic-mgmt/ansible/t1-8-tor.j2 on the
Ansible Controller.
    If you are using a module and expect the file to exist on
the remote, see the remote_src option
    fatal: [VM0304 -> ts102_tps_tb3(10.20.16.22)]: FAILED! => {
        "changed": false,
        "msg": "Could not find or access
't1-8-tor.j2'\nSearched

in:\n\t/data/private-sonic-mgmt/ansible/roles/eos/templates/t1-8-tor.j2\n\t/dat
a/private-sonic-mgmt/ansible/roles/eos/t1-8-tor.j2\n\t/data/private-sonic-mgmt/
ansible/roles/e

os/tasks/templates/t1-8-tor.j2\n\t/data/private-sonic-mgmt/ansible/roles/eos/ta
sks/t1-8-tor.j2\n\t/data/private-sonic-mgmt/ansible/templates/t1-8-tor.j2\n\t/d
ata/private-son

ic-mgmt/ansible/t1-8-tor.j2 on the Ansible Controller.\nIf
you are using a module and expect the file to exist on the remote, see the
remote_src option"
    }

```

In the case of T1 topologies we will need a tor template and a spine template.

```

None

ansible.errors.AnsibleFileNotFound: Could not find or access 't1-8-spine.j2'
ansible.errors.AnsibleFileNotFound: Could not find or access 't1-8-tor.j2'

```

Upon further investigation into the repository, most topologies are just symlinked to a standard eos template. Most of the t1 templates are symlinked to <https://github.com/nexthop-ai/private-sonic-mgmt/blob/master/ansible/roles/eos/templates/t1-lag-spine.j2> and <https://github.com/nexthop-ai/private-sonic-mgmt/blob/master/ansible/roles/eos/templates/t1-lag-tor.j2>

This includes topologies that are not using lags like Nexthop's default T1 topology. For example there is a template called t1-spine.j2 and t1-tor.j2 which are symlinked to t1-lag-spine.j2 and t1-lag-tor.j2 respectively.

Network Interface Architecture

The fundamental distinction between cEOS and vEOS lies in the network interface implementation:

- **cEOS:** Utilizes veth (virtual Ethernet) pairs exclusively
 - **Interface Separation:** One end resides within the Docker container's network namespace, while the other remains in the test server environment
 - **Topology Consistency:** Despite internal differences, the deployed testbed topology remains identical to vEOS configurations
-

Configuration Requirements

Inventory File Structure

The deployment playbook requires specific group configurations in the inventory file:

```
None
# Example: tps_lab_vms.yml
all:
  children:
    eos: # Essential for cEOS functionality
      children:
        vms_tps_tb1:
          vars:
            vm_console_base: 7100
        vms_tps_tb2:
          vars:
            vm_console_base: 7200
  servers:
```



```
children:
  server_ts107_tps_tb1:
  server_ts102_tps_tb2:
```

Deployment Process

Key Deployment Differences

The cEOS deployment process differs from traditional VM deployment in several important ways:

1. **Eliminated VM Startup:** The `start-vms` command is completely unnecessary
2. **Automated Container Creation:** Containers are generated during the `add-topo` process

Step-by-Step Deployment

```
Shell
# Navigate to the sonic-mgmt directory
cd ~/sonic-mgmt/ansible

# Deploy testbed with cEOS configuration
./testbed-cli.sh -t tps_testbed.yaml -m tps_lab_vms.yaml -k ceos
add-topo <testbed-name> password.txt

# Deploy Minigraph
./testbed-cli.sh -m tps_lab_vms.yaml -k ceos -t tps_testbed.yaml
deploy-mg <testbedname> tps_lab password.txt -vvv

# Verify successful deployment
docker ps | grep -E "(ceos_|net_)"
```

Useful links:

https://docs.google.com/document/d/1ebcviYw3hNqJymTRb03eampS_BnYGpLapWITZqS3nPk/edit?tab=t.thpk4knrcksc

<https://github.com/sonic-net/sonic-mgmt/blob/master/docs/testbed/README.testbed.Overview.md>